

AWS CloudFormation

Managing your infrastructure as code



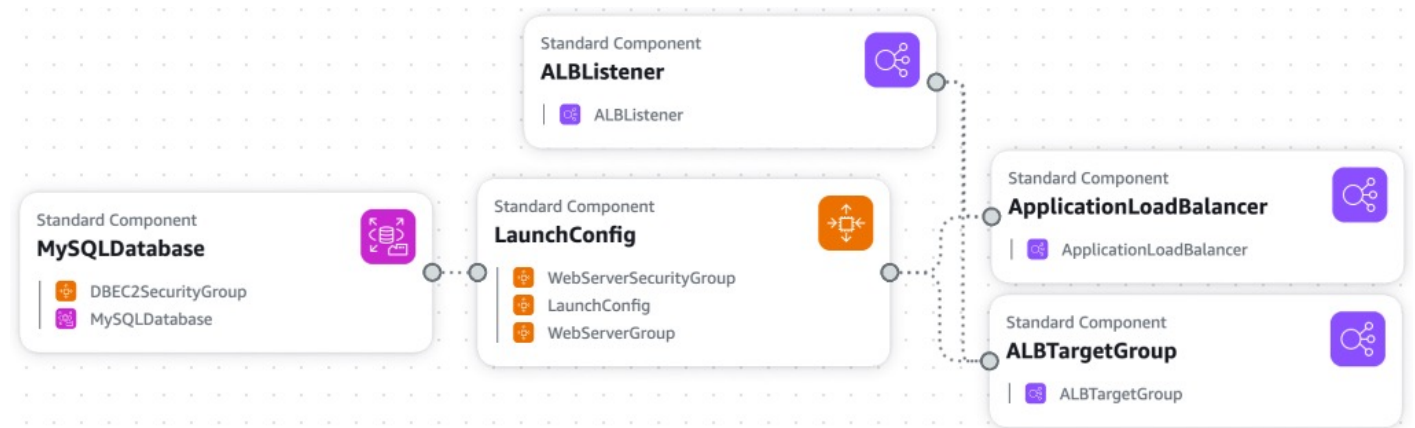
AWS CloudFormation

- CloudFormation is a declarative way of outlining your AWS Infrastructure, for any resources (most of them are supported)
- For example, within a CloudFormation template, you say:
 - I want a security group
 - I want two EC2 instances using this security group
 - I want two Elastic IPs for these EC2 instances
 - I want an S3 bucket
 - I want a load balancer (ELB) in front of these EC2 instances
- Then CloudFormation creates those for you, in the **right order**, with the **exact configuration** that you specify

CloudFormation – Template Example

```
1  AWSTemplateFormatVersion: '2010-09-09'
2  >Description: 'AWS CloudFormation Sample Template LAMP_Multi_AZ: C
10  >Parameters: ...
175 >Mappings: ...
481 Resources:
482   ApplicationLoadBalancer:
483     Type: AWS::ElasticLoadBalancingV2::LoadBalancer
484     Properties:
485       Subnets: !Ref Subnets
486   ALBListener:
487     Type: AWS::ElasticLoadBalancingV2::Listener
488     Properties:
489       DefaultActions:
490         - Type: forward
491           TargetGroupArn: !Ref ALBTargetGroup
492       LoadBalancerArn: !Ref ApplicationLoadBalancer
493       Port: '80'
494       Protocol: HTTP
495   ALBTargetGroup:
496     Type: AWS::ElasticLoadBalancingV2::TargetGroup
497     Properties:
498       HealthCheckIntervalSeconds: 10
499       HealthCheckTimeoutSeconds: 5
500       HealthyThresholdCount: 2
501       Port: 80
502       Protocol: HTTP
503       UnhealthyThresholdCount: 5
504       VpcId: !Ref VpcId
505       TargetGroupAttributes:
506         - Key: stickiness.enabled
507           Value: 'true'
```

Infrastructure Composer



Benefits of AWS CloudFormation (1/2)

- Infrastructure as code

- No resources are manually created, which is excellent for control
- The code can be version controlled for example using Git
- Changes to the infrastructure are reviewed through code

- Cost

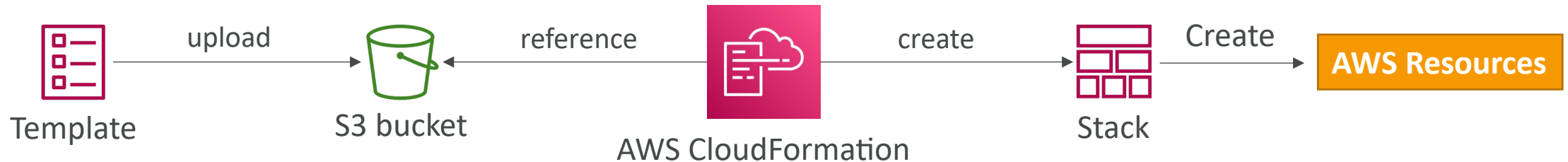
- Each resources within the stack is tagged with an identifier so you can easily see how much a stack costs you
- You can estimate the costs of your resources using the CloudFormation template
- Savings strategy: In Dev, you could automation deletion of templates at 5 PM and recreated at 8 AM, safely

Benefits of AWS CloudFormation (2/2)

- **Productivity**
 - Ability to destroy and re-create an infrastructure on the cloud on the fly
 - Automated generation of Diagram for your templates!
 - Declarative programming (no need to figure out ordering and orchestration)
- **Separation of concern: create many stacks for many apps, and many layers. Ex:**
 - VPC stacks
 - Network stacks
 - App stacks
- **Don't re-invent the wheel**
 - Leverage existing templates on the web!
 - Leverage the documentation

How CloudFormation Works

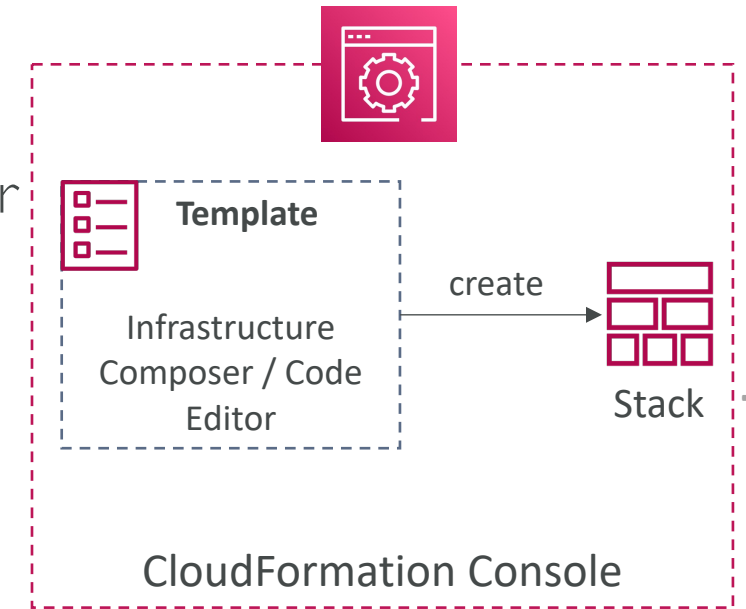
- Templates must be uploaded in S3 and then referenced in CloudFormation
- To update a template, we can't edit previous ones. We have to re-upload a new version of the template to AWS
- Stacks are identified by a name
- Deleting a stack deletes every single artifact that was created by CloudFormation.



Deploying CloudFormation Templates

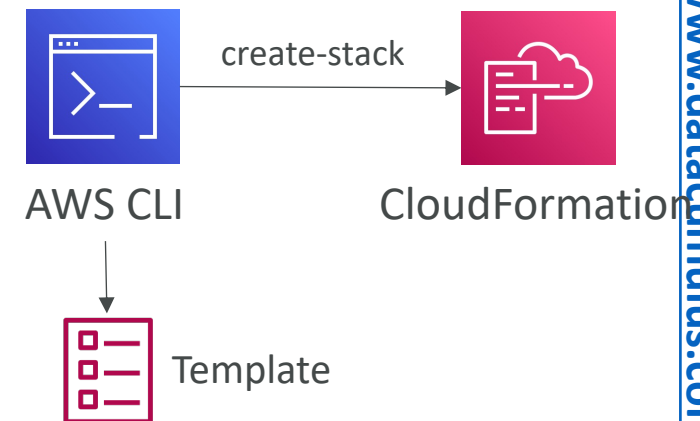
- **Manual way**

- Editing templates in Infrastructure Composer or code editor
- Using the console to input parameters, etc...
- We'll mostly do this way in the course for learning purposes



- **Automated way**

- Editing templates in a YAML file
- Using the AWS CLI (Command Line Interface) to deploy the templates, or using a Continuous Delivery (CD) tool
- Recommended way when you fully want to automate your flow



CloudFormation – Building Blocks

- Template's Components

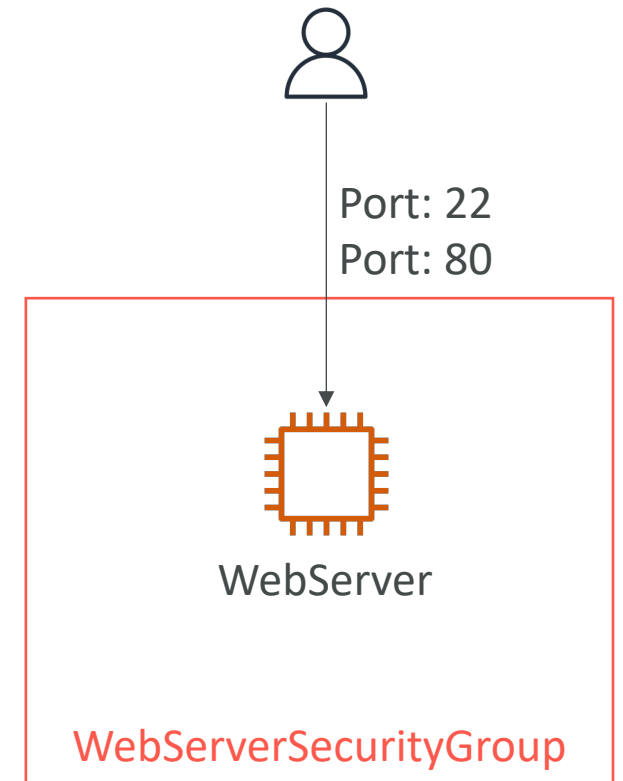
- `AWSTemplateFormatVersion` – identifies the capabilities of the template “2010-09-09”
- `Description` – comments about the template
- `Resources` (MANDATORY) – your AWS resources declared in the template
- `Parameters` – the dynamic inputs for your template
- `Mappings` – the static variables for your template
- `Outputs` – references to what has been created
- `Conditionals` – list of conditions to perform resource creation

- Template's Helpers

- `References`
- `Functions`

Introductory Example

- We're going to create a simple EC2 instance
 - And we're going to add security group to it
 - For now, forget about the code syntax
 - We'll look at the structure of the files later
-
- We'll see how in no-time, we are able to get started with CloudFormation!



YAML Crash Course

```
invoice: 34843
date: 2001-01-23
bill-to:
  given: Chris
  family: Dumars
  address:
    lines: |
      458 Walkman Dr.
      Suite #292
    city: Royal Oak
    state: MI
    postal: 48046
products:
  - sku: BL394D
    quantity: 4
    description: Basketball
    price: 450.00
  - sku: BL4438H
    quantity: 1
    description: Super Hoop
    price: 2392.00
```

- YAML and JSON are the languages you can use for CloudFormation
 - JSON is horrible for CF
 - YAML is great in so many ways
 - Let's learn a bit about it!
-
- Key value Pairs
 - Nested objects
 - Support Arrays
 - Multi line strings
 - Can include comments!

CloudFormation – Resources

- Resources are the core of your CloudFormation template (**MANDATORY**)
- They represent the different AWS Components that will be created and configured
- Resources are declared and can reference each other
- AWS figures out creation, updates and deletes of resources for us
- There are over 700 types of resources (!)
- Resource types identifiers are of the form:

service-provider** :: **service-name** :: **data-type-name

How do I find Resources documentation?

- I can't teach you all the 700+ resources, but I can teach you how to learn how to use them
- All the resources can be found here:
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-template-resource-type-ref.html>
- Then, we just read the docs 😊
- Example here (for an EC2 instance):
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-ec2-instance.html>

Analysis of CloudFormation Template

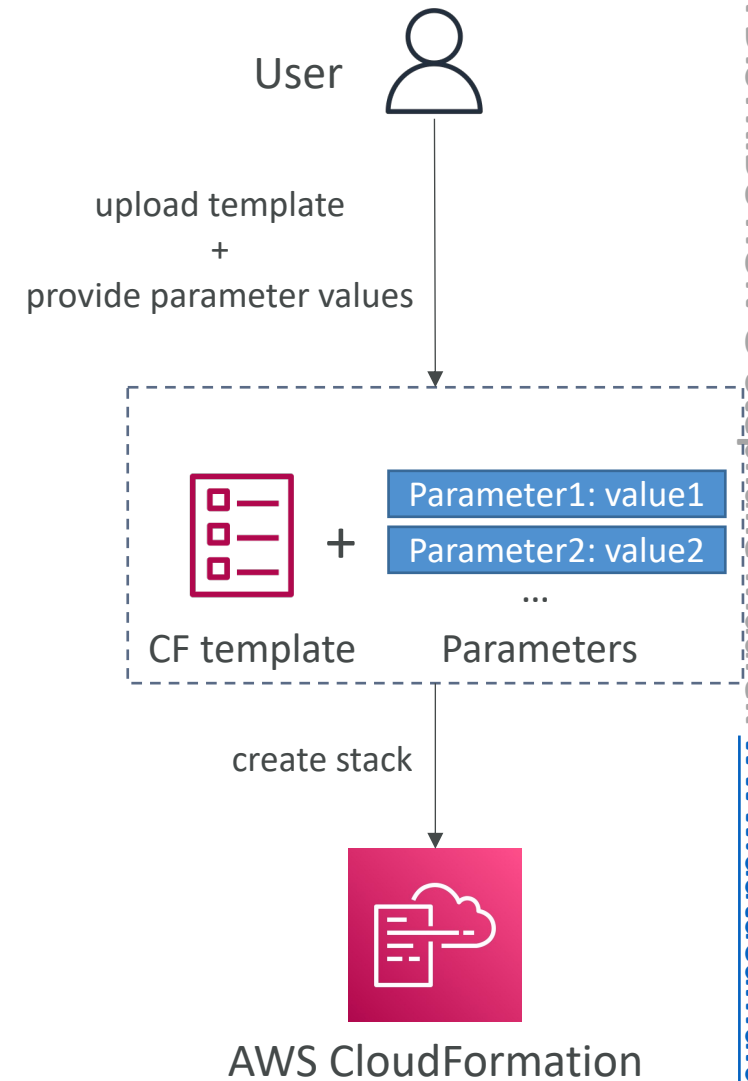
- Going back to the example of the introductory lecture, let's learn why it was written this way.
- Relevant documentation can be found here:
 - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-ec2-instance.html>
 - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-ec2-securitygroup.html>
 - <http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-ec2-eip.html>

CloudFormation – Resources FAQ

- Can I create a dynamic number of resources?
 - Yes, you can by using CloudFormation Macros and Transform
 - It is not in the scope of this course
- Is every AWS Service supported?
 - Almost. Only a select few niches are not there yet
 - You can work around that using CloudFormation Custom Resources

CloudFormation – Parameters

- Parameters are a way to provide inputs to your AWS CloudFormation template
- They're important to know about if:
 - You want to reuse your templates across the company
 - Some inputs can not be determined ahead of time
- Parameters are extremely powerful, controlled, and can prevent errors from happening in your templates, thanks to types



When should you use a Parameter?

Parameters:

SecurityGroupDescription:

Description: Security Group Description

Type: String

- Ask yourself this:
 - Is this CloudFormation resource configuration likely to change in the future?
 - If so, make it a parameter
- You won't have to re-upload a template to change its content 😊

CloudFormation – Parameters Settings

- Parameters can be controlled by all these settings:
 - Type:
 - String
 - Number
 - CommaDelimitedList
 - List<Number>
 - AWS-Specific Parameter (to help catch invalid values – match against existing values in the AWS account)
 - List<AWS-Specific Parameter>
 - SSM Parameter (get parameter value from SSM Parameter store)
 - Description
 - ConstraintDescription (String)
 - Min/MaxLength
 - Min/MaxValue
 - Default
 - AllowedValues (array)
 - AllowedPattern (regex)
 - NoEcho (Boolean)

CloudFormation – Parameters Example

AllowedValues

```
Parameters:
  InstanceType:
    Description: Choose an EC2 instance type
    Type: String
    AllowedValues:
      - t2.micro
      - t2.small
      - t2.medium
    Default: t2.micro

Resources:
  MyEC2Instance:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: !Ref InstanceType
      ImageId: ami-0c02fb55956c7d316
```

NoEcho

```
Parameters:
  DBPassword:
    Description: The database admin password
    Type: String
    NoEcho: true

Resources:
  MyDBInstance:
    Type: AWS::RDS::DBInstance
    Properties:
      DBInstanceClass: db.t2.micro
      AllocatedStorage: 20
      Engine: mysql
      MasterUsername: admin
      MasterUserPassword: !Ref DBPassword
      DBInstanceIdentifier: mydbinstance
```

How to Reference a Parameter?

```
Resources:
  DBSubnet1:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref MyVPC
```

- The `Fn::Ref` function can be leveraged to reference parameters
- Parameters can be used anywhere in a template
- The shorthand for this in YAML is `!Ref`
- The function can also reference other elements within the template

CloudFormation – Pseudo Parameters

- AWS offers us Pseudo Parameters in any CloudFormation template
- These can be used at any time and are enabled by default
- Important pseudo parameters:

Reference Value	Example Returned Value
AWS::AccountId	123456789012
AWS::Region	us-east-1
AWS::StackId	arn:aws:cloudformation:us-east-1:123456789012:stack/MyStack/1c2fa620-982a-11e3-aff7-50e2416294e0
AWS::StackName	MyStack
AWS::NotificationARNs	[arn:aws:sns:us-east-1:123456789012:MyTopic]
AWS::NoValue	Doesn't return a value

CloudFormation – Mappings

- Mappings are fixed variables within your CloudFormation template
- They're very handy to differentiate between different environments (dev vs prod), regions (AWS regions), AMI types...
- All the values are hardcoded within the template

Mappings:

Mapping01:

Key01:

Name: Value01

Key02:

Name: Value02

Key03:

Name: Value03

RegionMap:

us-east-1:

HVM64: ami-0ff8a91507f77f867

HVMG2: ami-0a584ac55a7631c0c

us-west-1:

HVM64: ami-0bdb828fd58c52235

HVMG2: ami-066ee5fd4a9ef77f1

eu-west-1:

HVM64: ami-047bb4163c506cd98

HVMG2: ami-0a7c483d527806435

Accessing Mapping Values (Fn::FindInMap)

- We use `Fn::FindInMap` to return a named value from a specific key
- `!FindInMap [ MapName, TopLevelKey, SecondLevelKey ]`

Mappings:

RegionMap:

us-east-1:

HVM64: `ami-0ff8a91507f77f867`

HVMG2: `ami-0a584ac55a7631c0c`

us-west-1:

HVM64: `ami-0bdb828fd58c52235`

HVMG2: `ami-066ee5fd4a9ef77f1`

Mappings work great for AMIs
Because AMIs are region-specific!

Resources:

MyEC2Instance:

Type: `AWS::EC2::Instance`

Properties:

ImageId: `!FindInMap [RegionMap, !Ref "AWS::Region", HVM64]`

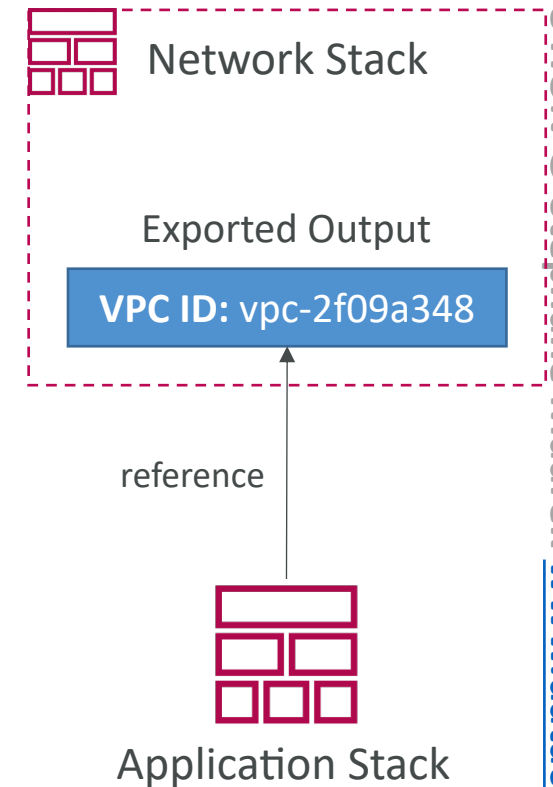
InstanceType: `t2.micro`

When would you use Mappings vs. Parameters?

- Mappings are great when you know in advance all the values that can be taken and that they can be deduced from variables such as
 - Region
 - Availability Zone
 - AWS Account
 - Environment (dev vs prod)
 - etc...
- They allow safer control over the template
- Use parameters when the values are really user specific

CloudFormation – Outputs

- The Outputs section declares *optional* outputs values that we can import into other stacks (if you export them first)!
- You can also view the outputs in the AWS Console or in using the AWS CLI
- They're very useful for example if you define a network CloudFormation, and output the variables such as VPC ID and your Subnet IDs
- It's the best way to perform some collaboration cross stack, as you let expert handle their own part of the stack



CloudFormation – Outputs

- Creating a SSH Security Group as part of one template
- We create an output that references that security group

Outputs:

StackSSHSecurityGroup:

Description: The SSH Security Group for our Company

Value: !Ref MyCompanyWideSSHSecurityGroup

Export:

Name: SSHSecurityGroup

CloudFormation – Outputs Cross-Stack Reference

- We then create a second template that leverages that security group
- For this, we use the `Fn::ImportValue` function
- You can't delete the underlying stack until all the references are deleted

Resources:

MySecureInstance:

Type: `AWS::EC2::Instance`

Properties:

ImageId: `ami-0742b4e673072066f`

InstanceType: `t2.micro`

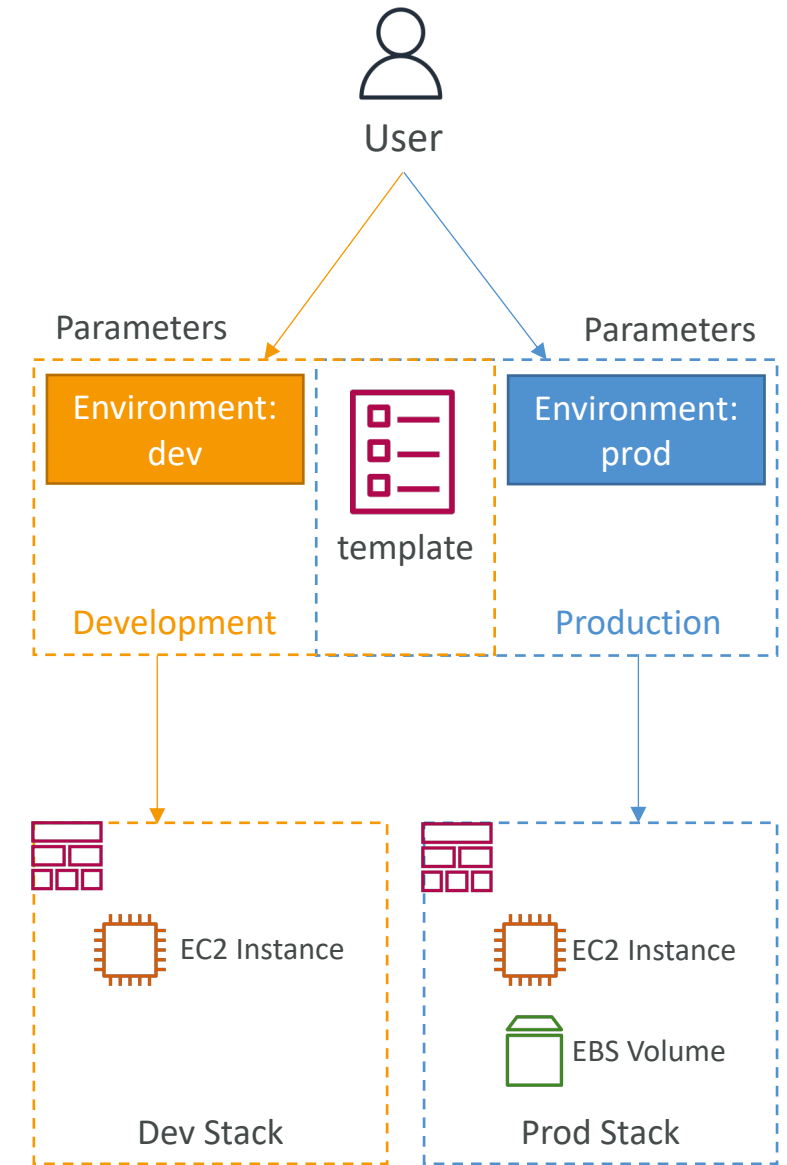
AvailabilityZone: `us-east-1a`

SecurityGroups:

– `!ImportValue SSHSecurityGroup`

CloudFormation – Conditions

- Conditions are used to control the creation of resources or outputs based on a condition
- Conditions can be whatever you want them to be, but common ones are:
 - Environment (dev / test / prod)
 - AWS Region
 - Any parameter value
- Each condition can reference another condition, parameter value or mapping



How to define a Condition

Conditions:

CreateProdResources: `!Equals [ !Ref EnvType, prod ]`

- The logical ID is for you to choose. It's how you name condition
- The intrinsic function (logical) can be any of the following:
 - Fn::And
 - Fn::Equals
 - Fn::If
 - Fn::Not
 - Fn::Or

How to use a Condition

- Conditions can be applied to resources / outputs / etc...

Resources:

MountPoint:

Type: `AWS::EC2::VolumeAttachment`

Condition: `CreateProdResources`

CloudFormation – Intrinsic Functions

Blue = must know

- Ref
- Fn::GetAtt
- Fn::FindInMap
- Fn::ImportValue
- Fn::Join
- Fn::Sub
- Fn::ForEach
- Fn::ToJsonString
- Condition Functions (Fn::If, Fn::Not, Fn::Equals, etc...)
- Fn::Base64
- Fn::Cidr
- Fn::GetAZs
- Fn::Select
- Fn::Split
- Fn::Transform
- Fn::Length

Intrinsic Functions – Fn::Ref

- The **Fn::Ref** function can be leveraged to reference
 - **Parameters** – returns the value of the parameter
 - **Resources** – returns the physical ID of the underlying resource (e.g., EC2 ID)
- The shorthand for this in YAML is **!Ref**

```
Resources:
  DBSubnet1:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref MyVPC
```

Intrinsic Functions – Fn::GetAtt

- Attributes are attached to any resources you create
- To know the attributes of your resources, the best place to look at is the documentation
- Example: the public DNS name of an EC2 instance!

Resources:

EC2Instance:

Type: `AWS::EC2::Instance`

Properties:

ImageId: `ami-0742b4e673072066f`

InstanceType: `t2.micro`

MyDNSRecord:

Type: `AWS::Route53::RecordSet`

Properties:

Name: `app.example.com`

ResourceRecords:

- `!GetAtt EC2Instance.PublicDnsName`

HostedZoneId: `Z123456ABCDEFG`

Type: `CNAME`

TTL: `300`

Intrinsic Functions – Fn::FindInMap

- We use `Fn::FindInMap` to return a named value from a specific key
- `!FindInMap [ MapName, TopLevelKey, SecondLevelKey ]`

Mappings:

RegionMap:

us-east-1:

HVM64: `ami-0ff8a91507f77f867`

HVMG2: `ami-0a584ac55a7631c0c`

us-west-1:

HVM64: `ami-0bdb828fd58c52235`

HVMG2: `ami-066ee5fd4a9ef77f1`

Resources:

MyEC2Instance:

Type: `AWS::EC2::Instance`

Properties:

ImageId: `!FindInMap [RegionMap, !Ref "AWS::Region", HVM64]`

InstanceType: `t2.micro`

Intrinsic Functions – Fn::ImportValue

- Import values that are exported in other stacks
- For this, we use the `Fn::ImportValue` function

Resources:

MySecureInstance:

Type: `AWS::EC2::Instance`

Properties:

ImageId: `ami-0742b4e673072066f`

InstanceType: `t2.micro`

AvailabilityZone: `us-east-1a`

SecurityGroups:

– `!ImportValue SSHSecurityGroup`

Intrinsic Functions – Fn::Base64

- Convert **String** to it's **Base64** representation

```
!Base64 "ValueToEncode"
```

- Example: pass encoded data to EC2 Instance's **UserData** property

```
Resources:
```

```
  WebServer:
```

```
    Type: AWS::EC2::Instance
```

```
    Properties:
```

```
      ...
```

```
      UserData:
```

```
        Fn::Base64: |  
          #!/bin/bash  
          dnf update -y  
          dnf install -y httpd
```

Intrinsic Functions – Condition Functions

Conditions:

CreateProdResources: `!Equals [ !Ref EnvType, prod ]`

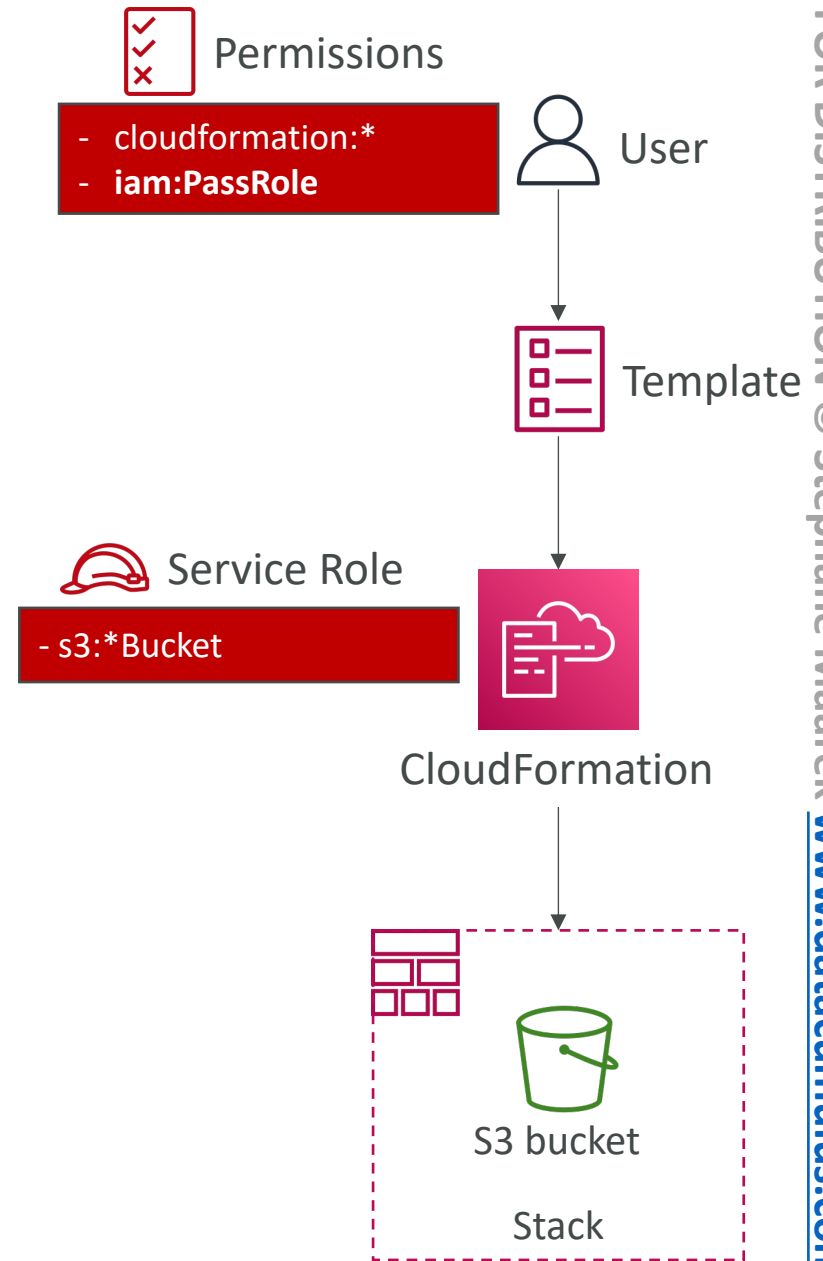
- The logical ID is for you to choose. It's how you name condition
- The intrinsic function (logical) can be any of the following:
 - Fn::And
 - Fn::Equals
 - Fn::If
 - Fn::Not
 - Fn::Or

CloudFormation – Rollbacks

- Stack Creation Fails:
 - Default: everything rolls back (gets deleted). We can look at the log
 - Option to disable rollback and troubleshoot what happened
- Stack Update Fails:
 - The stack automatically rolls back to the previous known working state
 - Ability to see in the log what happened and error messages
- Rollback Failure? Fix resources manually then issue **ContinueUpdateRollback API** from Console
 - Or from the CLI using continue-update-rollback API call

CloudFormation – Service Role

- IAM role that allows CloudFormation to create/update/delete stack resources on your behalf
- Give ability to users to create/update/delete the stack resources even if they don't have permissions to work with the resources in the stack
- Use cases:
 - You want to achieve the least privilege principle
 - But you don't want to give the user all the required permissions to create the stack resources
- User must have **iam:PassRole** permissions



CloudFormation Capabilities

- **CAPABILITY_NAMED_IAM** and **CAPABILITY_IAM**
 - Necessary to enable when your CloudFormation template is creating or updating IAM resources (IAM User, Role, Group, Policy, Access Keys, Instance Profile...)
 - Specify **CAPABILITY_NAMED_IAM** if the resources are named
- **CAPABILITY_AUTO_EXPAND**
 - Necessary when your CloudFormation template includes Macros or Nested Stacks (stacks within stacks) to perform dynamic transformations
 - You're acknowledging that your template may change before deploying
- **InsufficientCapabilitiesException**
 - Exception that will be thrown by CloudFormation if the capabilities haven't been acknowledged when deploying a template (security measure)

CloudFormation – DeletionPolicy Delete

- **DeletionPolicy:**
 - Control what happens when the CloudFormation template is deleted or when a resource is removed from a CloudFormation template
 - Extra safety measure to preserve and backup resources
- Default **DeletionPolicy=Delete**
 -  **Delete** won't work on an S3 bucket if the bucket is not empty

Resources:

MyEC2Instance:

Type: **AWS::EC2::Instance**

Properties:

ImageId: **ami-12345678**

InstanceType: **t2.micro**

KeyName: **my-key-pair**

SecurityGroupIds:

- **sg-12345678**

DeletionPolicy: Delete

Resources:

MyS3Bucket:

Type: **AWS::S3::Bucket**

DeletionPolicy: Delete



CloudFormation – DeletionPolicy Retain

- DeletionPolicy=Retain:
 - Specify on resources to preserve in case of CloudFormation deletes
- Works with any resources

Resources:

MyDynamoDBTable:

Type: `AWS::DynamoDB::Table`

Properties:

TableName: `MyTable`

AttributeDefinitions:

– AttributeName: `ID`

AttributeType: `S`

KeySchema:

– AttributeName: `ID`

KeyType: `HASH`

ProvisionedThroughput:

ReadCapacityUnits: `5`

WriteCapacityUnits: `5`

DeletionPolicy: `Retain`

CloudFormation – DeletionPolicy Snapshot

- DeletionPolicy=Snapshot
- Create one final snapshot before deleting the resource
- Examples of supported resources:
 - EBS Volume, ElastiCache Cluster, ElastiCache ReplicationGroup
 - RDS DBInstance, RDS DBCluster, Redshift Cluster, Neptune DBCluster, DocumentDB DBCluster

Resources:

MyDBInstance:

Type: `AWS::RDS::DBInstance`

Properties:

DBInstanceClass: `db.t2.micro`

AllocatedStorage: `20`

Engine: `mysql`

MasterUsername: `admin`

MasterUserPassword: `"ExamplePassword"`

DeletionPolicy: `Snapshot`

CloudFormation – Stack Policies

- During a CloudFormation Stack update, all update actions are allowed on all resources (default)
- A Stack Policy is a JSON document that defines the update actions that are allowed on specific resources during Stack updates
- Protect resources from unintentional updates
- When you set a Stack Policy, all resources in the Stack are protected by default
- Specify an explicit ALLOW for the resources you want to be allowed to be updated

```
{
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "Update:*",
      "Principal": "*",
      "Resource": "*"
    },
    {
      "Effect": "Deny",
      "Action": "Update:*",
      "Principal": "*",
      "Resource": "LogicalResourceId/ProductionDatabase"
    }
  ]
}
```

Allow updates on all resources
except the ProductionDatabase

CloudFormation – Termination Protection

- To prevent accidental deletes of CloudFormation Stacks, use TerminationProtection

CloudFormation – Custom Resources

- Used to
 - define resources not yet supported by CloudFormation
 - define custom provisioning logic for resources can that be outside of CloudFormation (on-premises resources, 3rd party resources...)
 - have custom scripts run during create / update / delete through Lambda functions (running a Lambda function to empty an S3 bucket before being deleted)
- Defined in the template using `AWS::CloudFormation::CustomResource` or `Custom::MyCustomResourceTypeName` (recommended)
- Backed by a Lambda function (most common) or an SNS topic

How to define a Custom Resource?

- **ServiceToken** specifies where CloudFormation sends requests to, such as Lambda ARN or SNS ARN (required & must be in the same region)
- Input data parameters (optional)



Resources:

MyCustomResourceUsingLambda:

Type: Custom::MyLambdaResource

Properties:

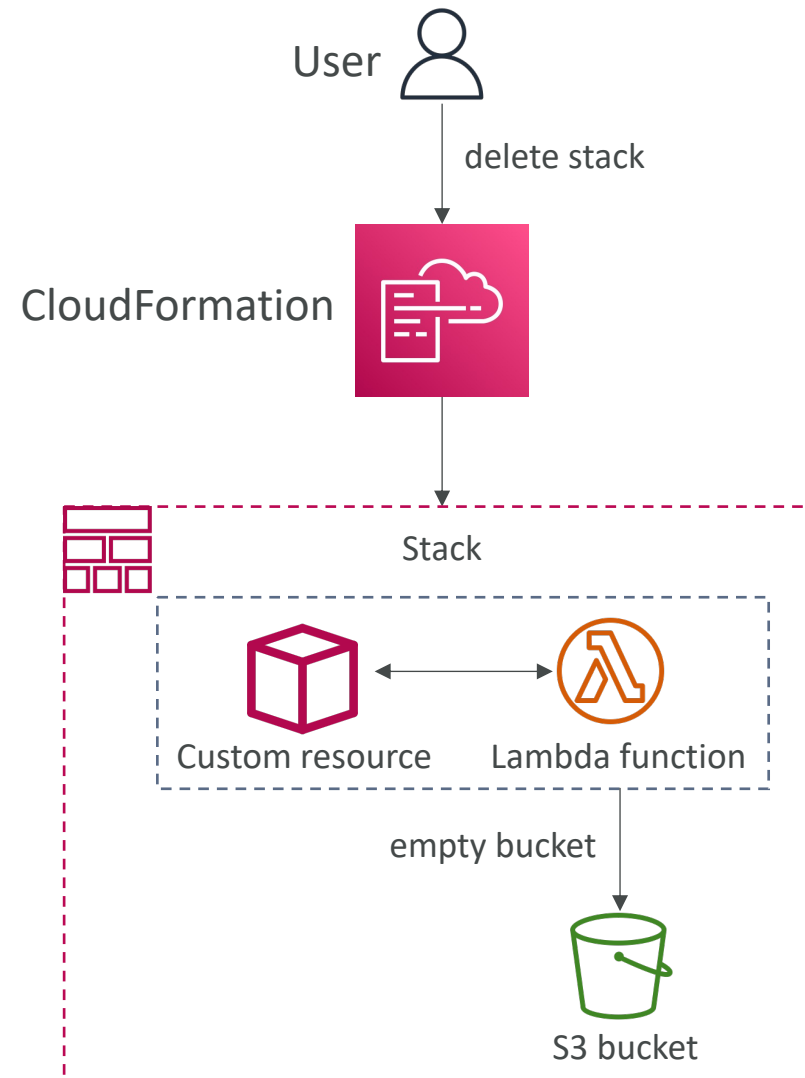
ServiceToken: arn:aws:lambda:REGION:ACCOUNT_ID:function:FUNCTION_NAME

Input values (optional)

ExampleProperty: "ExampleValue"

Use Case – Delete content from an S3 bucket

- You can't delete a non-empty S3 bucket
- To delete a non-empty S3 bucket, you must first delete all the objects inside it
- We can use a custom resource to empty an S3 bucket before it gets deleted by CloudFormation



CloudFormation – StackSets

- Create, update, or delete stacks across **multiple accounts and regions** with a single operation/template
- Target accounts to create, update, delete stack instances from StackSets
- When you update a stack set, *all* associated stack instances are updated throughout all accounts and regions
- Can be applied into all accounts of an **AWS Organization**
- Only Administrator account (or Delegated Administrator) can create StackSets

